

Model-Based Scheduling Analysis for Software Projects

Frank Padberg and David Weiss

Department of Computer Science
Saarland University
Saarbrücken, Germany
padberg@cs.uni-saarland.de

Abstract— We show how to compute optimal policies for the scheduling of software development projects under uncertainty. Our approach is based on a stochastic scheduling model that explicitly captures the strong feedback between the tasks in software development (“ripple effects”). We apply reinforcement learning to the optimization problem. For a selected sample project, we compute the optimal policy, simulate the project, and analyze the task assignments that are made by the optimal policy. From the analysis of the simulated schedules, we derive tentative, generic scheduling guidelines. The guidelines prioritize certain tasks based on the characteristics of the software architecture, and assign tasks according to the past performance of the developers.

Keywords: *software project scheduling, software process models, software cost estimation, software life cycle, reinforcement learning*

I. INTRODUCTION

When planning a software development project, the exact duration of the tasks is not known in advance and varies largely. Software components typically cannot be developed independently of each other since they are coupled through the software architecture. Hence, changes in one part of the software often cause changes in other parts. This kind of feedback is well-known as the “ripple effect” in software development.

In this paper, we present a scheduling model for software projects that takes uncertainty and ripple effects explicitly into account. Task completion times are *stochastic* in the model. Design changes occur randomly, and the amount of rework caused by them (that is, the size of the ripple effect) is stochastic, too.

We use the model to compute *optimal scheduling policies* for projects under uncertainty. The policies are dynamic in the sense that they base their task assignment decisions on the current project situation. As the model has a huge state space even for small instances, we apply *reinforcement learning* to compute the optimal policy. This allows us to optimize medium-sized projects.

To illustrate our approach in the paper, we compute the optimal policy for a hypothetical sample project. We analyze the scheduling decisions of the optimal policy in detail. Then we derive some very initial scheduling guidelines from the results.

II. STATE OF THE ART

Stochastic scheduling models in *operations research* typically avoid feedback between concurrent tasks, because feedback makes the mathematics and proofs much more complicated. Some models do allow feedback, but limit the feedback to cyclic work within the tasks, not between tasks [1] [2]. This currently is the state of the art in stochastic scheduling [3]. We cannot dispense with explicit feedback between tasks in our scheduling model, because this is a key characteristic of the software development process. In this respect, our stochastic, feedback-centered model is unique.

There is surprisingly little research in *software engineering* concerned with software project scheduling. The well-known system dynamics model [4] considers total work force and project completion time, but not at the level of scheduling individual tasks. There is a large literature on cost estimation and effort distribution, but not on detailed schedules. We know of only one modeling approach partially similar to ours, and it focuses on the testing part of the software process [5].

We have presented our model in detail [6]. Hence, we only sketch the model in this paper. In previous work, we could only optimize very small instances, mainly serving illustrative purposes [7]. The machine learning-based schedule optimization and the detailed analysis of a larger instance in the current paper are novel.

III. SOFTWARE PROCESS MODEL

A. States and Transitions

In our process model, a software project moves from state to state. The *state* contains three elements:

- The net progress vector tells for each component how much development time has been spent on the component so far, excluding any rework. This captures the net progress of the project.
- The countdown value expresses how much time is left until the deadline of the project is exceeded. After the deadline, the project will be canceled.
- The rework vector counts for each component how much rework still has to be served before net progress is possible again. All rework is caused by design changes that propagate through the project (“ripple effects”).

Fig. 1 shows a cut-out of a sample process. There are four components involved, denoted “A” through “D”. Two teams are working on the project, denoted “1” and “2”. Currently, the project is in state ζ . Net progress has been achieved on three of the four components so far; f.e., two time slices for component “A” (first entry in progress vector (2, 1, 3, 0)). Currently, there is no rework (rework vector (0, 0, 0, 0)). There are 16 time units left until the deadline is reached. Note that time is *discrete* in the model; one slice could represent, say, a week or a month, depending on the desired granularity and the amount of historical data available.

Transitions between states are triggered in two cases: If a team finishes its current component, work force becomes available and requires rescheduling by the manager; or, if some problem occurs that requires changes to the software’s design, rescheduling again is necessary to clear the rework caused by the changes. Hence, in the model a *scheduling action* occurs in each state. Actions simply are assignments of teams to tasks. Fig. 1 shows two of the possible actions in state ζ . F.e., the first action assigns team 1 to component A and team 2 to component B. Actions are subject to the *scheduling policy* that is in place.

After scheduling, the project moves to its next state probabilistically. That is, each possible successor state occurs only with a certain *transition probability* that gets computed from empirical data about the past projects, see below.

Fig. 1 shows one possible next state η : Three time slices of net progress have passed, after which team 1 has finished its component A, indicated by an infinity symbol. The net progress of component B has increased by 3 slices. The countdown has been reduced by three slices. Team 1 now can be reassigned to some other component. Other outcomes are possible as well, with varying probability. Eventually, all components are finished, or the project exceeds its deadline and is canceled.

Each transition between states is associated with some *transition cost*. Different notions of cost can be used in our model, in particular, staffing cost. In this paper, though, we focus on *duration*; that is, the “cost” of a task equals its completion time specified in time slices.

Formally, the project model is a Markov decision process (MDP). This is a well-known class of models in operations research [10].

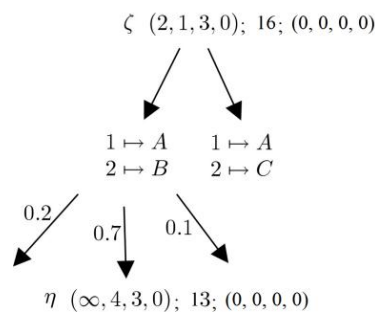


Figure 1. Cut-out of a sample software process

B. Input Data

To compute the transition probabilities in the model, three kinds of input data are required: base probabilities, rework probabilities, and coupling degrees.

Base probabilities specify the distribution of the completion time for each component and team. For example, in the distribution shown in Fig. 2, there is a 10 per cent probability that the corresponding component will finish after 5 time slices, a 30 per cent probability that it will finish after 6 time slices, and so on.

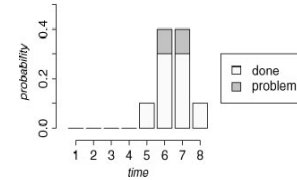


Figure 2. Sample base probabilities (“large component”)

The base probabilities also specify how likely it is that design problems (and, hence, ripple effects and rework) will originate from the component at certain points in time. For example, in Fig. 2 there is a 10 per cent probability that this component will trigger rework after 6 time slices. Fig. 3 shows a similar distribution that corresponds to a “small” component.

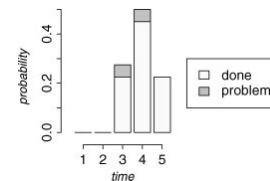


Figure 3. Sample base probabilities (“small component”)

The base probabilities reflect a number of well-known drivers for the development time required for a component, including its complexity and riskiness. The larger and more complex a component, the larger the mean of the distribution; the more risky a component, the larger the variance of the distribution. The base probabilities also capture the experience of the team working on the component; since the distributions are computed from historical project data, they also capture the past performance of this (and similar) teams when working on a component of this type. The experience and past performance of the team enter the distribution the same way as the complexity and riskiness of the component.

Note that in our model, a component is either finished or not. The model does not assume any particular development methodology for the components; any characteristics of the methodology get reflected in the base distributions.

In addition, for each component there is a probability distribution for the amount of *rework* assuming that the component is affected by rework. In this paper, we assume that reworking a “small” component takes one time slice,

and reworking a “large” component takes one or two time slices with equal probability.

The *coupling degrees* specify the likelihood that problems propagate through the software. For each set of components K , they specify the probability that the set M of components will be affected by rework if a design problem occurs in the set of components K . F.e, $\alpha(\{B,C\},\{B,C,D\})$ specifies the likelihood of ripple effects in components B, C, and D, assuming that there was a problem in B and C. Typically, components in which a problem occurred will be affected by rework. Rework may affect components that were already finished, but not components that have not been worked on yet.

The coupling degrees reflect characteristics of the software architecture, in particular, the strength of the coupling between the software components. They get computed from the architectural design and from empirical data about the propagation of design changes in similar software systems.

C. Specialization

The base probabilities allow for specifying that a team is *specialized* on certain kind of component. In such a case, the base distribution has a smaller mean and variance compared to a non-specialized team. In this paper, we use left shifts of the original base distributions to capture specialization (shifted by two time slices for “small,” and three time slices for “large” components). For example, Fig. 4 shows the base distribution for “small” components when a specialized team is working on the component.

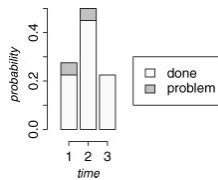


Figure 4. Base distribution for specialized teams (“small” component)

IV. OPTIMIZATION USING MACHINE LEARNING

The path taken by the software MDP depends on the scheduling actions chosen in each state. A mapping that specifies one scheduling action for each state is called a *policy*. When the policy is given, the MDP turns into a standard Markov process that has an expected “cost” – recall that “cost” in this paper means *project duration*. The optimization problem is apparent now: Compute a policy that has minimal expected project duration.

In principle, this optimization problem can be solved using dynamic programming [9][10]. Whether this is feasible in practice depends on the size of the MDP, that is, the number of its states. Our software MDP grows rapidly with

the size of the project. The number of states depends on several factors:

- The number of components and teams. The more components, the larger the state vector.
- The support (“size”) of the base distributions. The more time slices it can take before a component is finished, the larger the number of possible states.
- The choice of time unit. The more fine-grained the model reflects real time, the larger the number of states.
- The coupling degree. Minimal and maximal coupling induce fewer states than uniform (or other intermediate forms of) coupling.

Even for small instances built from base distributions as above, the software MDP typically has several million reachable states. Therefore, exact schedule optimization using dynamic programming in the software MDP is feasible only for instances with about 5 components. For larger instances, the state space is just too huge.

Instead of dynamic programming, we use *reinforcement learning* (RL) to optimize the policies in our software MDP [11][12]. Reinforcement learning is solidly based on dynamic programming (DP) principles and has been successfully applied to other scheduling problems before [13][14]. Roughly, the expensive exact computation of expected cost values for the states in DP is substituted by computationally much cheaper simulations of the process, yielding approximations of the expected cost values. In addition, the simulations focus on the states that are most likely to occur and do not visit many rare states. This explains the speedup and memory savings as compared to DP.

We have pointed out the potential of a simulation-based approach for this class of optimization problems in an earlier paper [8].

Starting from a random policy, RL iteratively produces a sequence of policies that tend to improve quickly. Under mild assumptions, these policies are known to converge to an optimal policy [15]. In practice, RL is stopped when the next policy doesn’t improve much over the previous one. In many applications, the result is often close to optimal. For the software MDP, whenever we were able to compute an exact optimal policy, the near-optimal policy computed by RL was in fact optimal.

V. SIMULATION EXPERIMENT

A. Procedure

In our simulation experiments based on the stochastic model, we generally proceed as follows. We fix a particular project instance, including the number of components and teams, the base probabilities for each team and component (with or without specialization), and the coupling degrees. As input, we use the distributions shown in section III.

Next, we compute the optimal scheduling policy for the instance using reinforcement learning.

Once the optimal policy has been computed, we simulate the project 500,000 times under the optimal policy. This yields not only a good approximation of the optimal expected project duration, but also a large number of paths (“trajectories”) that the project can take when being scheduled with the optimal policy. In particular, we can observe the task assignments made by the optimal policy in a variety of project states.

Finally, we analyze the scheduling decisions made by the policy during the simulations.

Our computations were carried out on a standard PC with 4 cores running at 2.6 GHz, and 12 GB of main memory. Our RL optimization library is a pure object-oriented design that we implemented in Java. With this equipment, we currently optimize instances with up to 14 components, depending on the coupling.

The computing time for the running example in this paper with 8 components (see Table 1) is about 15 minutes. The reinforcement learning algorithm visits about 2 million states during the optimization. We used the specific algorithm Sarsa(λ) [11]. A large value of 0.98 for the decay parameter λ worked best for this instance.

B. Average Schedules

To analyze the behavior of the optimal policy for a given project instance, we employ a tool called *average schedules*. They get computed as follows.

For each individual simulation of the project path (“run”) it is straightforward to determine its schedule, that is, which components have been assigned to which team, and in which order. The runs now are grouped according to schedule. Since the runs in each group share the same assignments, it makes sense to compute for each component the mean of its completion times in the simulations. Finally, the schedule annotated with the mean completion times is depicted as a standard Gantt chart.

Fig. 5 shows such an average schedule. Components A, F, and D are assigned to team 1, in this order; G, B, and E are assigned to team 2; H and C are assigned to team 3. We use the notation $(AFD)_1(GBE)_2(HC)_3$ for this schedule. The schedule occurred in 17.0 per cent of all simulations and resulted in a mean project duration of 9.1 time slices. The mean of the completion time of, say, component F, gets computed as 2.6 time slices from the simulations.

To make the schedule more informative, the chart also shows for each component the number of the specialized teams in parentheses.

The average schedules capture and summarize the typical paths that the project will take under the given policy. They help with analyzing task assignments and task ordering phenomena in the simulated projects. For small instances, up to a few dozen average schedules occur; for larger instances, the number of average schedules can be substantial (in the hundreds). In this paper, we focus on the average schedules occurring with the highest relative frequency in the simulations.

C. A Sample Project

In this paper, we study the optimal policy for a sample project that has eight components, six small ones (A to F) and two large ones (G and H), according to the distributions from section III. There are three teams (1 to 3).

Coupling is assumed to be *uniform*. This means that each set of components is equally likely to be affected by ripple effects than any other. This kind of coupling reflects a very symmetric architecture, or a lack of knowledge about the strength of the coupling between the components. We found uniform coupling to be a very useful tool for studying ripple effects.

Specialization (refer to subsection III.C) is mixed in the project. Each component has a specialized team. Some components have more than one specialized team (D and F each have two, E has three), but for the large components, only one team is specialized, see Table 1. If a specialized team works on a large component, the base distribution from Fig. 2 is shifted to the left by three time slices. There is one small component E that can be developed equally fast by all three teams.

TABLE I. SPECIALIZATION IN THE “8:3 UNIF” INSTANCE

component	A	B	C	D	E	F	G	H
specialized teams	1	2	3	1,2	1,2,3	1,3	2	3

D. The Optimal Policy

The most frequent average schedule of the optimal policy for this project setting is shown in Fig. 5.

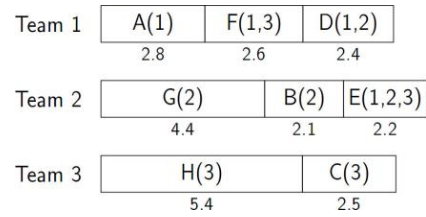


Figure 5. 8:3 unif, schedule $(AFD)_1(GBE)_2(HC)_3$ with $p=17.0\%$, $c=9.1$

The optimal policy assigns the two large components G and H early in the project to the corresponding specialized teams 2 and 3. Team 1 starts with a small component. All teams in the sequel of the project get assigned to small components for which they are specialized. In particular, B and C are assigned to team 2 and 3, respectively. Any other assignment would violate the specialization and likely lead to a project delay. This particular schedule occurs in 17 per cent of the simulations.

There is a similar average schedule where H is completed faster than G, and, hence, E is assigned to team 3 instead of team 2, see Fig. 6. This schedule occurs with a frequency of 15 per cent.



Figure 6. 8:3 unif, schedule (AFD)₁(GB)₂(HCE)₃ with $p=14.9\%$, $c=9.4$

In yet another case, team 1 makes quick progress relative to the other teams, and ends up handling four instead of three small components, see Fig. 7.

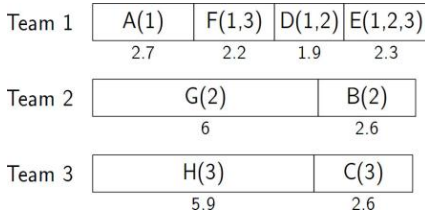


Figure 7. 8:3 unif, schedule (AFDE)₁(GB)₂(HC)₃ with $p=7.8\%$, $c=9.5$

There are similar cases with the order of F and D switched, accounting for another 29 per cent of the simulations of the optimal policy.

It can also happen that components G and H get finished fairly quickly, so that teams 2 and 3 finally work on three components each, see Fig. 8.

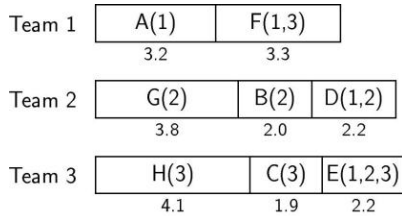


Figure 8. 8:3 unif, schedule (AF)₁(GBD)₂(HCE)₃ with $p=2.8\%$, $c=8.7$

Even when scheduling with the optimal policy, the project can take an unfavorable path. For example, Fig. 9 shows a (rare) average schedule where component G takes so long to complete that team 3 ends up handling three components instead of two, including a component that it is *not* specialized for. This results in a late project that needs about 20 per cent more time to complete than the typical case.

The “8:3 unif” instance exhibits 680 different average schedules. We have analyzed all of them, but can show only the most interesting ones in this paper. Only 13 average schedules occur with a frequency of 1 per cent or more. The 21 most frequent schedules account for more than 90 per cent of the simulations.

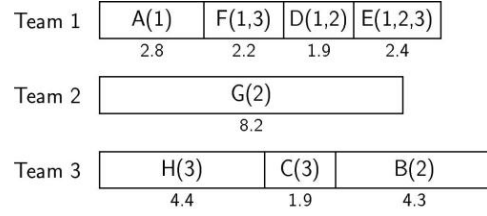


Figure 9. 8:3 unif, schedule (AFDE)₁(G)₂(HCB)₃ with $p=0.2\%$, $c=10.8$

E. Rework

Since the components are coupled through the software architecture, changes in one component can lead to rework in other components (“ripple effect”). The amount of rework in the project depends on the strength of the coupling and on the probability of the occurrence of “design problems” as specified by the base probability distributions.

In the “8:3 uniform” project instance, a substantial amount of rework occurs in the schedules. Fig. 10 shows the most frequent average schedule from Fig. 5 again, now with the rework for each component indicated as a shaded area at the end of the corresponding bar.

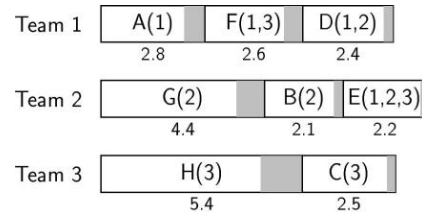


Figure 10. 8:3 unif, most frequent schedule, rework marked

For example,, about 20 per cent of the total completion time for the large component H arises from rework. This rework is caused by feedback from the other tasks. This “unplanned” extra effort could not have been estimated properly at the project start from the input distributions and would have flawed any initial plan for a project schedule.

F. Static versus Dynamic Scheduling

One might ask whether one could achieve an optimal schedule by fixing some “smart” scheduling plan in advance. This would save the computational effort of a full MDP optimization. One simple heuristic (that is not uncommon in real software projects) fixes the order in which the components will be assigned to the teams. The policy based on such a “priority list” is called a *list policy*. Whenever a team finishes its current component, it is assigned to the next component in the list. List policies are static in the sense that the priority list is fixed at the start of the project and doesn’t change during the project.

The best list policy for the “8:3 unif” project is AGHFBCDE and was computed by exhaustive search. The best list policy gets the initial assignment right, assigning the three components A, G, and H the same way as the optimal policy. Yet, it occurs in only 11 per cent of the simulations that the best list policy assigns *all* components in an optimal way. In more than 50 per cent of the simulations, the best list policy assigns two or more of the remaining components to the wrong team. This results in substantial project delays of up to 20 per cent as compared to the optimal policy. Clearly, there is a lot to gain when scheduling *dynamically*, taking into account the path that the project has taken so far.

VI. CONCLUSIONS

We presented a stochastic scheduling model for software development projects. We applied reinforcement learning to compute an optimal scheduling policy for a medium-sized sample instance.

We analyzed the most typical schedules of the optimal policy for this instance, in particular, the task assignments made by the optimal policy in the course of the project. We visualized the task assignments using average schedules.

When comparing the results for this paper’s project instance with a number of other instances that we have optimized, we found that the optimal policies make similar decisions. We try and derive some general – yet highly tentative – properties of “good” scheduling policies from the behavior of the optimal policies that we have studied so far:

- A good policy balances large and small components among the teams. This way the teams will finish at about the same time, which seems to lead to the shortest project duration.
- A good policy exploits any specialization of the teams when assigning tasks. A specialized team on average will finish its component faster than a non-specialized team. This seems to be a key factor for minimizing the project completion time.
- A good policy assigns tasks based on the current progress and likely future path of the project. It seems

that scheduling under uncertainty requires abandoning the idea of sticking to a pre-computed schedule.

We currently carry out more case studies and check under which conditions these guidelines hold.

REFERENCES

- [1] K. Neumann: *Stochastic Project Networks*, Lecture Notes in Economics and Mathematical Systems 344, Springer, 1990
- [2] R. H. Möhring: “Scheduling under uncertainty: Optimizing against a randomizing adversary.” Proc. 3rd Int. Workshop on Approximation Algorithms for Combinatorial Optimization Problems, LNCS 1913, pp. 15–26, Springer, 2000
- [3] W. Herroelen, R. Leus: “Project scheduling under uncertainty: Survey and research potentials.” European Journal of Operational Research EJOR, vol. 165, issue 2 (2005) pp. 289–306
- [4] T. K. Abdel-Hamid, Madnick, S. E.: *Software Project Dynamics*. Prentice Hall, 1991
- [5] D. M. Raffo, Kellner, M. I.: “Modeling Software Processes Quantitatively and Evaluating the Performance of Process Alternatives.” in: *Elements of Software Process Assessment and Improvement*. ed. K. El Emam, Madhavji, N. H., IEEE Computer Society Press, 1999, pp. 297–341
- [6] F. Padberg: “A Discrete Simulation Model for Assessing Software Project Scheduling Policies.” Int. J. on Software Process Improvement and Practice SPIP, vol. 7, issue 3 (2002) pp. 127–139
- [7] F. Padberg: “A Study on Optimal Scheduling of Software Projects.” Int. J. on Software Process Improvement and Practice SPIP, vol. 11, issue 2 (2006) pp. 77–91
- [8] F. Padberg: “On the Potential of Process Simulation in Software Project Schedule Optimization.” Int. Workshop on Software Cybernetics IWSC at COMPSAC (2005) pp. 127–130
- [9] D. Bertsekas: *Dynamic Programming and Optimal Control*. Athena Scientific, 1995
- [10] R. Bellman: *Dynamic Programming*. Princeton University Press, 1957
- [11] R. Sutton, Barto, A.: *Reinforcement Learning. An Introduction*. MIT Press, 1998
- [12] D. Bertsekas, Tsitsiklis, J.: *Neuro-Dynamic Programming*. Athena Scientific, 1996
- [13] R. Crites, Barto, A.: “Improving elevator performance using reinforcement learning.” Proc. Annual Conf. on Advances in Neural Information Processing Systems NIPS (1996) pp. 1017–1023
- [14] W. Zhang, Dietterich, T.: “A reinforcement learning approach to job-shop scheduling.” Proc. 14th Int. Joint Conf. on Artificial Intelligence IJCAI (1995) pp. 1014–1025
- [15] P. Dayan: “The convergence of TD(λ) for general λ .” Machine Learning vol. 8, issue 3–4 (1992) pp. 341–362